



UNIVERSIDAD TECNOLÓGICA DE QUERÉTARO

Nombre de la Organización:

UNIVERSIDAD TECNOLÓGICA DE QUERÉTARO (UTEQ)

Presenta:

VICTOR GUADALUPE LOPEZ CEBREROS

Grupo:

IDGS17

Unidad 2. Práctica 3. Logs - Parte 1

Implementación de Logging Estructurado con Serilog y Centralización de Eventos en Seq

Los valores numéricos, fragmentos de log y resultados de consulta consignados en este reporte provienen de ejecuciones reales de la aplicación y de consultas directas a la API de Seq. No se trata de estimaciones ni de reconstrucciones posteriores.

1. Datos generales

Campo	Valor
Proyecto	VulnerableApp, rama secure
Framework	ASP.NET Core MVC sobre .NET 10
Biblioteca de logging	Serilog
Serilog.AspNetCore	10.0.0
Serilog.Sinks.Console	6.1.1
Serilog.Sinks.File	7.0.0
Serilog.Sinks.Seq	9.1.0
Serilog.Enrichers.Environment	3.0.1
Serilog.Enrichers.Thread	4.0.0
Servidor de agregación	Seq, imagen datalust/seq:latest en Docker
Endpoint de ingesta	http://localhost:5341
Interfaz web de consulta	http://localhost:8081
Controladores instrumentados	HomeController, SearchController, AuthController, CommentController, ApiController
Ruta de persistencia en disco	VulnerableApp/Logs/log-YYYYMMDD.txt
Política de rotación	Diaria (RollingInterval.Day)

2. Objetivo y alcance

Implementar una estrategia de registro de eventos estructurado sobre una aplicación ASP.NET Core, sustituyendo el proveedor de logging predeterminado del framework por Serilog, y centralizar la totalidad de los eventos generados en un servidor Seq desplegado en contenedor, de modo que resulte posible consultar, filtrar y correlacionar la actividad de la aplicación mediante propiedades y no únicamente mediante búsqueda textual.

El alcance comprende la instrumentación de los cinco controladores de la aplicación con registro de entrada, salida, duración, identidad del solicitante, dirección de origen, parámetros de la petición, advertencias de seguridad, errores y excepciones, así como la verificación empírica —con tráfico real— de que los controles implementados operan según lo previsto y de que ningún dato sensible alcanza los registros.

3. Marco conceptual

3.1 Logging estructurado frente a logging textual

El registro tradicional produce cadenas de texto planas en las que los datos variables quedan interpolados dentro del mensaje. Su consulta posterior depende de búsquedas por coincidencia de subcadenas, técnica frágil ante cambios de redacción y que no permite agregaciones ni filtros sobre los valores individuales.

El logging estructurado, en cambio, conserva el mensaje como **plantilla** y los valores como **propiedades nombradas e independientes**. La instrucción `_logger.LogInformation("Inicio Auth.Login Usuario:{User} IP:{IP}", username, ip)` no produce una cadena ya resuelta, sino un evento con la plantilla y dos propiedades tipadas, `User` e `IP`. Esta representación permite que el destino de almacenamiento indexe cada propiedad por separado y habilita consultas del tipo «todos los eventos donde `User` sea un valor determinado», imposibles de expresar de forma confiable sobre texto plano.

Esta distinción es la que justifica la introducción de Serilog frente al proveedor predeterminado de `Microsoft.Extensions.Logging`, y la que otorga sentido a la incorporación de un servidor de agregación como Seq: sin estructura, la centralización se reduciría a concentrar archivos de texto en un mismo lugar.

3.2 Sinks y enrichers

Dos abstracciones organizan la arquitectura de Serilog y conviene precisarlas antes de describir la configuración.

Un **sink** es un destino de escritura. Serilog admite múltiples sinks simultáneos sobre un mismo evento, y cada uno puede aplicar su propio nivel mínimo y su propio

formato de salida. La escritura ocurre en paralelo desde el punto de vista lógico, de modo que un mismo evento puede quedar simultáneamente impreso en consola, persistido en disco y transmitido por red.

Un **enricher** es un componente que agrega propiedades adicionales a todo evento que atraviesa el pipeline, con independencia del punto del código donde se haya originado. Su valor consiste en incorporar contexto que el desarrollador no necesita repetir en cada llamada: el nombre del host, el identificador del hilo de ejecución o el nombre de la aplicación se adjuntan de forma transversal y automática.

4. Desarrollo

4.1 Incorporación de paquetes

Se agregaron al proyecto los paquetes correspondientes al núcleo de Serilog para ASP.NET Core, a los tres sinks requeridos y a los dos enrichers adicionales:

```
dotnet add package Serilog.AspNetCore
dotnet add package Serilog.Sinks.Console
dotnet add package Serilog.Sinks.File
dotnet add package Serilog.Sinks.Seq
dotnet add package Serilog.Enrichers.Environment
dotnet add package Serilog.Enrichers.Thread
```

4.2 Despliegue de Seq en contenedor

Seq se desplegó como contenedor con reinicio automático, exponiendo el puerto de ingesta y el de la interfaz web:

```
docker run --name seq -d --restart unless-stopped ^
-e ACCEPT_EULA=Y ^
-e SEQ_FIRSTRUN_NOAUTHENTICATION=True ^
-p 5341:80 -p 8081:80 ^
datalust/seq:latest
```

Hallazgo de configuración. El comando indicado en la guía original de la práctica —idéntico al anterior salvo por la ausencia de la variable `SEQ_FIRSTRUN_NOAUTHENTICATION`— **no logra arrancar el contenedor** en la versión actual de la imagen. Desde una revisión reciente, Seq exige que en el primer arranque se defina una contraseña de administrador mediante `SEQ_FIRSTRUN_ADMINPASSWORD` o bien se desactive explícitamente la autenticación mediante `SEQ_FIRSTRUN_NOAUTHENTICATION`. Sin ninguna de las dos, el proceso interno aborta durante la inicialización con el error No default

admin password was supplied, y la directiva --restart unless-stopped provoca que el contenedor entre en un ciclo indefinido de reinicio, permaneciendo en estado Restarting sin llegar nunca a atender peticiones.

Este comportamiento constituye una decisión deliberada del fabricante orientada a evitar despliegues sin autenticación por omisión, y resulta coherente con el principio de configuración segura por defecto. La desactivación aplicada en esta práctica es admisible únicamente por tratarse de un entorno local de laboratorio sin exposición a red; en cualquier despliegue accesible desde otros equipos debería definirse la contraseña de administrador, dado que Seq concentra información detallada sobre la actividad interna de la aplicación.

4.3 Configuración del punto de entrada

Se adoptó el patrón de **bootstrap logger** recomendado por la documentación oficial de Serilog, en lugar de la configuración en línea propuesta en la guía. El patrón consiste en crear un logger mínimo antes de construir el host, sustituirlo por la configuración definitiva una vez que el contenedor de dependencias está disponible, y envolver toda la ejecución en un bloque de manejo de excepciones.

```
csharp
using Microsoft.EntityFrameworkCore;
using Microsoft.Extensions.Hosting;
using Serilog;
using VulnerableApp.Data;

Log.Logger = new LoggerConfiguration()
    .WriteTo.Console()
    .CreateBootstrapLogger();

Log.Information("Iniciando VulnerableApp...");

try
{
    var builder = WebApplication.CreateBuilder(args);

    builder.Host.UseSerilog((context, services, configuration) => configuration
```

```
.ReadFrom.Configuration(context.Configuration)
.ReadFrom.Services(services)
.Enrich.FromLogContext();

builder.Services.AddControllersWithViews();
builder.Services.AddDbContext<AppDbContext>(options =>

options.UseSqlServer(builder.Configuration.GetConnectionString("DefaultConnecti
on")));
builder.Services.AddSession();

var app = builder.Build();

if (!app.Environment.IsDevelopment())
{
    app.UseExceptionHandler("/Home/Error");
    app.UseHsts();
}

app.UseHttpsRedirection();
app.UseSerilogRequestLogging();
app.UseRouting();
app.UseSession();
app.UseAuthorization();
app.MapStaticAssets();
app.MapControllerRoute(
    name: "default",
    pattern: "{controller=Home}/{action=Index}/{id?}").WithStaticAssets();

app.Run();
```

```

}
catch (Exception ex) when (ex is not HostAbortedException)
{
    Log.Fatal(ex, "VulnerableApp termino inesperadamente durante el arranque");
}
finally
{
    Log.CloseAndFlush();
}

```

Tres decisiones de esta configuración merecen justificación explícita.

Bootstrap logger. Una configuración de logging construida únicamente después de `builder.Build()` deja sin cobertura toda la fase previa de arranque. Cualquier excepción producida durante el registro de servicios, la lectura de configuración o la resolución de la cadena de conexión ocurriría antes de que exista un logger, y se perdería sin registro alguno. El bootstrap logger cubre esa ventana y es reemplazado de forma transparente por la configuración definitiva una vez que esta queda disponible.

Posición de `UseSerilogRequestLogging()`. Este middleware condensa en un único evento resumido lo que el registro predeterminado de ASP.NET Core distribuye en varias líneas por petición, incorporando método, ruta, código de estado y duración. Su ubicación temprana en el pipeline resulta necesaria para que la medición de duración comprenda el tiempo consumido por los middlewares posteriores; situarlo al final produciría mediciones sistemáticamente inferiores a la latencia real percibida por el cliente.

Exclusión de `HostAbortedException`. El filtro `when (ex is not HostAbortedException)` responde a una condición detectada durante la propia práctica. Las herramientas de línea de comandos de Entity Framework Core construyen el host de la aplicación para inspeccionar los servicios registrados y, acto seguido, lo abortan de forma deliberada mediante esta excepción, que constituye por tanto un mecanismo de control de flujo previsto y no una falla. Sin la exclusión, cada ejecución de `dotnet ef database update` producía un evento de nivel Fatal en los registros, contaminando el historial con falsos positivos de máxima severidad. El detalle de este hallazgo se documenta en la sección 7.

`Log.CloseAndFlush()` en el bloque `finally`. Los sinks de archivo y de red operan con escritura almacenada en búfer por razones de rendimiento. Sin el vaciado

explícito al terminar, los últimos eventos generados —precisamente los más relevantes cuando la aplicación finaliza de forma anómala— podrían no alcanzar su destino.

4.4 Configuración externa del comportamiento del logger

La totalidad del comportamiento de Serilog se declaró en `appsettings.json`, quedando el código libre de configuración de destinos y niveles:

`json`

```
"Serilog": {
  "MinimumLevel": {
    "Default": "Information",
    "Override": {
      "Microsoft": "Warning",
      "Microsoft.AspNetCore": "Warning",
      "Microsoft.EntityFrameworkCore": "Warning"
    }
  },
  "WriteTo": [
    { "Name": "Console" },
    { "Name": "File", "Args": { "path": "Logs/log-.txt", "rollingInterval": "Day" } },
    { "Name": "Seq", "Args": { "serverUrl": "http://localhost:5341" } }
  ],
  "Enrich": [ "FromLogContext", "WithMachineName", "WithThreadId" ],
  "Properties": { "Application": "VulnerableApp" }
}
```

El archivo `appsettings.Development.json` sobrescribe únicamente los niveles, dejando intactos sinks y enrichers:

`json`

```
"Serilog": {
  "MinimumLevel": {
    "Default": "Debug",
    "Override": {
```

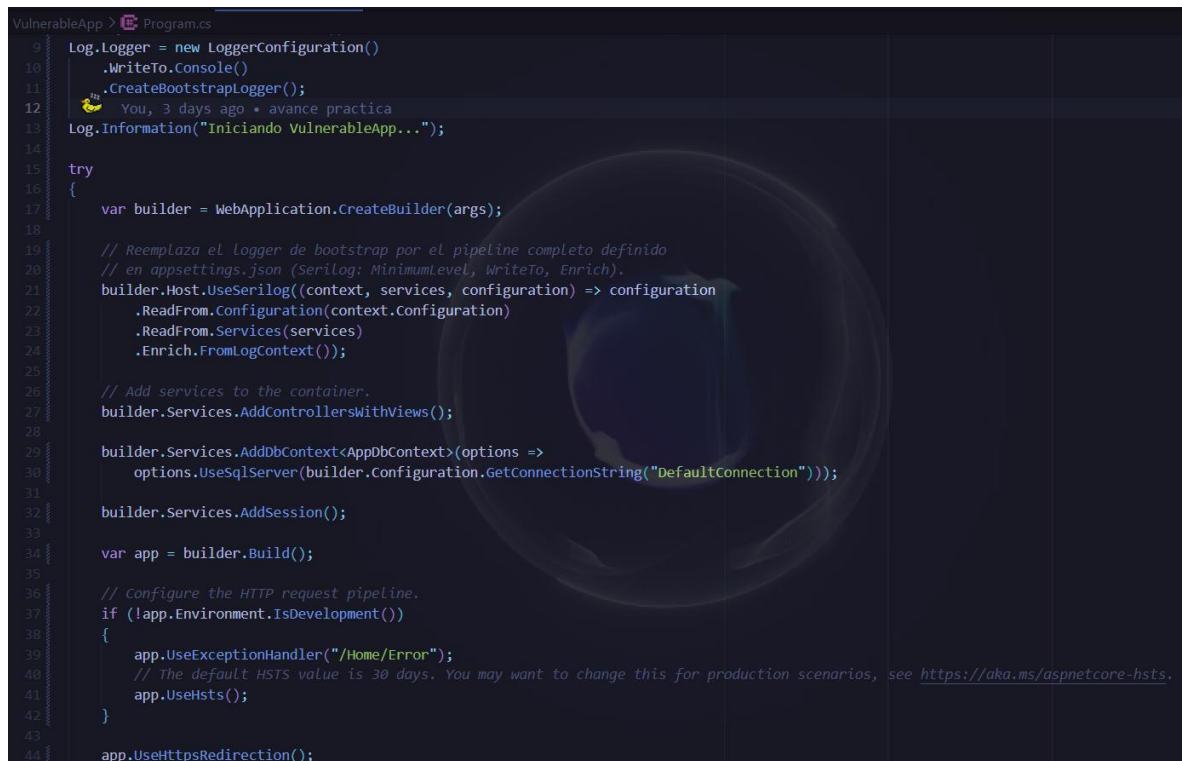
```

    "Microsoft": "Information",
    "Microsoft.AspNetCore": "Warning",
    "Microsoft.EntityFrameworkCore": "Information"
}
}
}

```

Los **overrides por espacio de nombres** cumplen una función determinante para la utilidad del registro. Sin ellos, el nivel Information aplicado de forma global provocaría que el framework y Entity Framework Core emitieran un volumen elevado de eventos —cada consulta SQL generada, cada resolución de ruta— que sepultaría los eventos propios de la aplicación. Elevar el umbral de esos espacios de nombres a Warning conserva sus advertencias y errores, que sí resultan pertinentes, mientras suprime el ruido informativo. La configuración de desarrollo relaja selectivamente el umbral de Entity Framework Core a Information, nivel en el que sí registra las consultas ejecutadas, lo cual resulta deseable durante la depuración pero no en operación normal.

La propiedad Properties.Application adjunta un identificador constante a todo evento emitido. En un servidor Seq que recibe registros de varias aplicaciones, esta propiedad es la que permite segmentar el flujo por origen.



```

VulnerableApp > Program.cs
9  Log.Logger = new LoggerConfiguration()
10     .WriteTo.Console()
11     .CreateBootstrapLogger();
12     You, 3 days ago • avance practica
13     Log.Information("Iniciando VulnerableApp...");
14
15     try
16     {
17         var builder = WebApplication.CreateBuilder(args);
18
19         // Reemplaza el logger de bootstrap por el pipeline completo definido
20         // en appsettings.json (Serilog: MinimumLevel, WriteTo, Enrich).
21         builder.Host.UseSerilog((context, services, configuration) => configuration
22             .ReadFrom.Configuration(context.Configuration)
23             .ReadFrom.Services(services)
24             .Enrich.FromLogContext());
25
26         // Add services to the container.
27         builder.Services.AddControllersWithViews();
28
29         builder.Services.AddDbContext<AppDbContext>(options =>
30             options.UseSqlServer(builder.Configuration.GetConnectionString("DefaultConnection")));
31
32         builder.Services.AddSession();
33
34         var app = builder.Build();
35
36         // configure the HTTP request pipeline.
37         if (!app.Environment.IsDevelopment())
38         {
39             app.UseExceptionHandler("/Home/Error");
40             // The default HSTS value is 30 days. You may want to change this for production scenarios, see https://aka.ms/aspnetcore-hsts.
41             app.UseHsts();
42         }
43
44         app.UseHttpsRedirection();

```

4.5 Instrumentación de los controladores

Los cinco controladores requeridos reciben ILogger<T> mediante inyección de dependencias y registran, en cada acción:

- **Entrada a la acción**, en nivel Information, incluyendo identidad del solicitante y dirección IP de origen.
- **Salida de la acción**, en nivel Information, incorporando la duración en milisegundos medida con Stopwatch.
- **Parámetros de la petición**, con exclusión deliberada de todo campo de credencial.
- **Advertencias**, en nivel Warning: intento de autenticación fallido, acceso sin sesión establecida, acceso a recurso ajeno y envío de comentario vacío.
- **Errores**, en nivel Error, mediante LogError(ex, ...) con la excepción como primer argumento, de modo que Serilog capture la traza de pila como propiedad estructurada y no como texto concatenado en el mensaje.
- **Eventos de autenticación** con el prefijo convencional AuthEvent: seguido del tipo de evento: LoginSuccess, LoginFailed, Logout, AccesoNoAutenticado y AccesoNoAutorizado.

La adopción de un prefijo común para los eventos de autenticación constituye una decisión de diseño orientada a la consulta posterior. Permite recuperar la totalidad de la actividad relacionada con el control de acceso mediante un único filtro, con independencia del controlador que la haya originado, y establece una convención que un equipo puede extender de forma consistente.

Fragmento representativo del registro de un intento fallido de autenticación:

```
csharp
```

```
_logger.LogInformation("Inicio Auth.Login (POST) Usuario:{User} IP:{IP}",  
username, ip);
```

```
...
```

```
_logger.LogWarning("AuthEvent:LoginFailed Usuario:{User} IP:{IP}  
Motivo:CredencialesInvalidas",  
username, ip);
```

Obsérvese que el mensaje registra el nombre de usuario y el motivo del rechazo, pero **en ningún caso la contraseña suministrada**. El nombre de usuario es necesario para detectar patrones de ataque —intentos repetidos contra una misma cuenta o contra cuentas distintas desde una misma dirección—, mientras que la

contraseña carece por completo de utilidad diagnóstica y su registro constituiría una exposición de credenciales en texto plano dentro de un archivo que, por su naturaleza, suele replicarse hacia sistemas de respaldo y de agregación con controles de acceso más laxos que los de la base de datos original.

```

_logger.LogInformation("Inicio Auth.Dashboard Usuario:{User} IP:{IP}", user, ip);

var userId = HttpContext.Session.GetInt32("UserId");
if (!userId.HasValue)
{
    _logger.LogWarning("AuthEvent:AccesoNoAutenticado a Dashboard IP:{IP}", ip);
    return RedirectToAction("Login");
}

try
{
    var dbUser = _db.Users.Find(userId.Value);

    sw.Stop();
    _logger.LogInformation(
        "Fin Auth.Dashboard Usuario:{User} DuracionMs:{DurationMs}", user, sw.ElapsedMilliseconds);

    return View(dbUser);
}
catch (Exception ex)
{
    _logger.LogError(ex, "Error en Auth.Dashboard Usuario:{User} IP:{IP}", user, ip);
    throw;
}
}

```

5. Retos resueltos

5.1 Función de cada sink

Sink	Función técnica	Justificación de su inclusión
Console	Escribe cada evento en la salida estándar del proceso, durante el desarrollo local. Es el primer con formato legible y coloreado por nivel.	Proporciona retroalimentación inmediata al ejecutar la aplicación y no depende de ningún servicio externo.
File	Persiste los eventos en archivos de texto en disco, reinicio del proceso y a la indisponibilidad aplicando rotación diaria del servidor de agregación. Constituye el conforme al patrón log-respaldo de última instancia y la base de YYYYMMDD.txt.	Ofrece un historial que sobrevive al reinicio del proceso y a la indisponibilidad del servidor de agregación. Constituye la auditoría.
Seq	Transmite los eventos en formato JSON estructurado	Habilita búsqueda, filtrado y correlación por propiedades individuales, así como agregaciones y alertas, capacidades

Sink	Función técnica	Justificación de su inclusión
	(CLEF) al servidor mediante HTTP, con envío por lotes.	inalcanzables mediante búsqueda textual sobre archivos planos.

La combinación responde a una lógica de redundancia deliberada. Los tres sinks presentan modos de falla distintos: la consola desaparece al cerrar el proceso, el archivo depende de la disponibilidad del disco y de la política de rotación, y el sink de red depende de que el servidor esté operativo. Ante la caída de Seq, el sink correspondiente descarta o encola los eventos según su configuración, pero los sinks de consola y archivo continúan operando sin interrupción, de modo que ningún evento se pierde por completo.

5.2 Control del nivel mínimo desde configuración

El requisito se satisfizo mediante la instrucción `configuration.ReadFrom.Configuration(context.Configuration)` dentro de `UseSerilog`, que delega la totalidad de la configuración del logger al sistema de configuración de ASP.NET Core. Este sistema resuelve el valor efectivo aplicando la cadena de proveedores en orden de precedencia: `appsettings.json` como base, `appsettings.{Environment}.json` como sobrescritura según el valor de `ASPNETCORE_ENVIRONMENT`, y las variables de entorno por encima de ambos.

La consecuencia práctica es que el nivel de detalle del registro puede modificarse en un despliegue sin recompilar ni redistribuir la aplicación, capacidad relevante en operación real: ante un incidente en producción, elevar temporalmente el detalle a Debug constituye una herramienta diagnóstica de primer orden. La verificación empírica de este comportamiento se documenta en la sección 6.7.

5.3 Enrichers adicionales y su justificación

Más allá de `FromLogContext`, requerido por la práctica, se incorporaron dos enrichers adicionales.

WithMachineName adjunta el nombre del host que generó el evento, `LAPTOP-1M1UUQDQ` en este entorno. Su justificación aparece en despliegues con más de una instancia —contenedores replicados, máquinas virtuales tras un balanceador de carga—, donde permite determinar en qué nodo se originó cada evento sin necesidad de inspeccionar los registros de cada servidor por separado. Resulta particularmente valioso ante fallas que se manifiestan en una sola instancia, escenario en el que la propiedad convierte un problema difícil de reproducir en una consulta directa.

WithThreadId adjunta el identificador del hilo que procesó la petición. ASP.NET Core atiende peticiones concurrentes sobre un pool de hilos, de modo que los eventos de varias peticiones simultáneas quedan entrelazados en el flujo de salida. La marca temporal, aun con resolución de milisegundos, no siempre permite separar dos ejecuciones concurrentes, mientras que el identificador de hilo constituye un discriminante inequívoco para reconstruir la secuencia correspondiente a una misma ejecución.

Corresponde precisar una limitación de este último enriquecer en modelos asíncronos: dado que una operación async puede reanudarse en un hilo distinto al que la inició, el identificador no permanece constante a lo largo de toda una petición. Para correlación estricta extremo a extremo, la construcción adecuada es un identificador de correlación propagado mediante LogContext, mecanismo que FromLogContext habilita y que el registro automático de peticiones de Serilog aprovecha internamente.

6. Pruebas ejecutadas y resultados

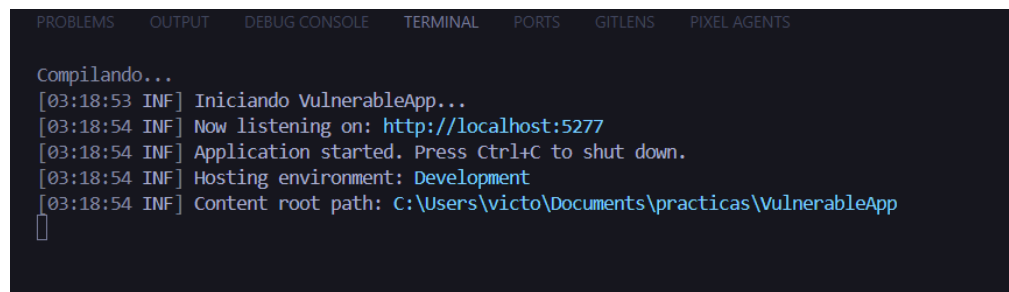
6.1 Tráfico de prueba generado

Los cinco controladores se ejercitaron mediante curl, cubriendo la página principal, búsqueda con y sin término, autenticación fallida y exitosa, acceso al panel con y sin sesión, alta de comentario válido y vacío, y los tres escenarios de la API: respuesta 401 sin autenticación, respuesta 200 sobre el recurso propio y respuesta 403 ante el intento de acceso al recurso de otro usuario.

Esta cobertura fue diseñada de forma deliberada para ejercitar **rutas de éxito y rutas de rechazo en igual proporción**. Un conjunto de pruebas limitado a peticiones válidas produciría un registro compuesto exclusivamente por eventos de nivel Information, dejando sin verificar precisamente la instrumentación de advertencias y errores, que es la de mayor valor operativo.

6.2 Evidencia del sink de consola

La ejecución de la aplicación produce en la terminal el flujo de eventos con formato legible y prefijos de nivel abreviados, intercalando eventos propios de la aplicación y eventos automáticos del middleware de registro de peticiones.

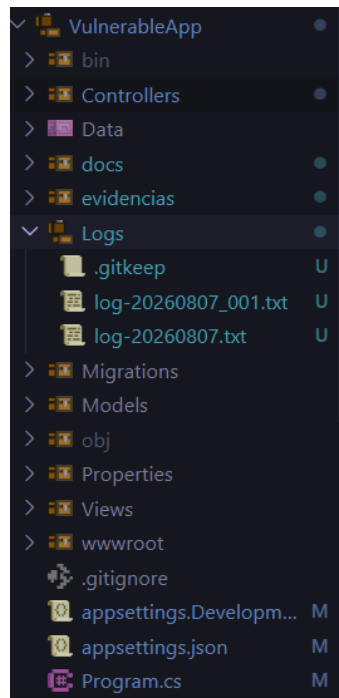


```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  GITLENS  PIXEL AGENTS

Compilando...
[03:18:53 INF] Iniciando VulnerableApp...
[03:18:54 INF] Now listening on: http://localhost:5277
[03:18:54 INF] Application started. Press Ctrl+C to shut down.
[03:18:54 INF] Hosting environment: Development
[03:18:54 INF] Content root path: C:\Users\victo\Documents\practicas\VulnerableApp
█
```

6.3 Evidencia del sink de archivo

El sink de archivo crea el directorio Logs durante la primera escritura y genera un archivo por día conforme al patrón de rotación configurado.



```
VulnerableApp > Logs > log-20260807.txt
1 2026-08-07 03:02:58.002 -06:00 [INF] Now listening on: http://localhost:5277
2 2026-08-07 03:02:58.024 -06:00 [INF] Application started. Press Ctrl+C to shut down.
3 2026-08-07 03:02:58.024 -06:00 [INF] Hosting environment: Development
4 2026-08-07 03:02:58.024 -06:00 [INF] Content root path: C:\Users\Victo\Documents\practicas\VulnerableApp
5 2026-08-07 03:03:14.894 -06:00 [WRN] Failed to determine the https port for redirect.
6 2026-08-07 03:03:15.138 -06:00 [INF] Inicio Search.Index Usuario:anonimo IP:::1 Parametros:{"search":"admin"}
7 2026-08-07 03:03:15.139 -06:00 [DBG] Ejecutando consulta parametrizada Users.Where(Username contains admin)
8 2026-08-07 03:03:15.530 -06:00 [WRN] No store type was specified for the decimal property 'Balance' on entity type 'User'. This will cause values to be s
9 2026-08-07 03:03:16.585 -06:00 [INF] Executed DbCommand (230ms) [Parameters=[@search_contains=?' (Size = 4000)], CommandType="Text", CommandTimeout='3
10 SELECT [u].[Id], [u].[Balance], [u].[CreatedAt], [u].[Email], [u].[PasswordHash], [u].[Username]
11 FROM [Users] AS [u]
12 WHERE [u].[Username] LIKE @search_contains ESCAPE N'\
13 2026-08-07 03:03:16.685 -06:00 [INF] Fin Search.Index Usuario:anonimo Resultados:1 DuracionMs:1553
14 2026-08-07 03:03:16.833 -06:00 [INF] HTTP GET /Search/Index responded 200 in 1934.9568 ms
15 2026-08-07 03:03:16.933 -06:00 [INF] Inicio Auth.Login (POST) Usuario:admin IP:::1
16 2026-08-07 03:03:16.975 -06:00 [INF] Executed DbCommand (11ms) [Parameters=[@username=?' (Size = 4000)], CommandType="Text", CommandTimeout='30']
17 SELECT TOP(1) [u].[Id], [u].[Balance], [u].[CreatedAt], [u].[Email], [u].[PasswordHash], [u].[Username]
18 FROM [Users] AS [u]
19 WHERE [u].[Username] = @username
20 2026-08-07 03:03:16.977 -06:00 [DBG] Verificando hash BCrypt para Usuario:admin (password nunca se registra en logs)
21 2026-08-07 03:03:17.175 -06:00 [INF] AuthEvent:LoginSuccess Usuario:admin IP:::1 DuracionMs:242
22 2026-08-07 03:03:17.180 -06:00 [INF] HTTP POST /Auth/Login responded 302 in 274.1799 ms
23 2026-08-07 03:03:17.307 -06:00 [INF] Inicio Api.GetUser IP:::1 Parametros:{"id":1}
24 2026-08-07 03:03:17.307 -06:00 [DBG] Comparando UsuarioId solicitado:1 con UsuarioId de sesion:1
25 2026-08-07 03:03:17.431 -06:00 [INF] Executed DbCommand (7ms) [Parameters=[@p=?' (DbType = Int32)], CommandType="Text", CommandTimeout='30']
26 SELECT TOP(1) [u].[Id], [u].[Balance], [u].[CreatedAt], [u].[Email], [u].[PasswordHash], [u].[Username]
27 FROM [Users] AS [u]
28 WHERE [u].[Id] = @p
29 2026-08-07 03:03:17.431 -06:00 [INF] Fin Api.GetUser UsuarioId:1 DuracionMs:125
30 2026-08-07 03:03:17.456 -06:00 [INF] HTTP GET /api/user/1 responded 200 in 165.2294 ms
31 2026-08-07 03:03:37.635 -06:00 [INF] Inicio Home.Index Usuario:anonimo IP:::1
32 2026-08-07 03:03:37.636 -06:00 [INF] Fin Home.Index Usuario:anonimo DuracionMs:2
33 2026-08-07 03:03:37.655 -06:00 [INF] HTTP GET / responded 200 in 29.3866 ms
34 2026-08-07 03:03:37.741 -06:00 [INF] Inicio Home.Privacy Usuario:anonimo IP:::1
35 2026-08-07 03:03:37.741 -06:00 [INF] Fin Home.Privacy Usuario:anonimo DuracionMs:0
36 2026-08-07 03:03:37.754 -06:00 [INF] HTTP GET /Home/Privacy responded 200 in 14.6526 ms
37 2026-08-07 03:03:37.856 -06:00 [INF] Inicio Search.Index Usuario:anonimo IP:::1 Parametros:{"search":"admin"} 🐛
```

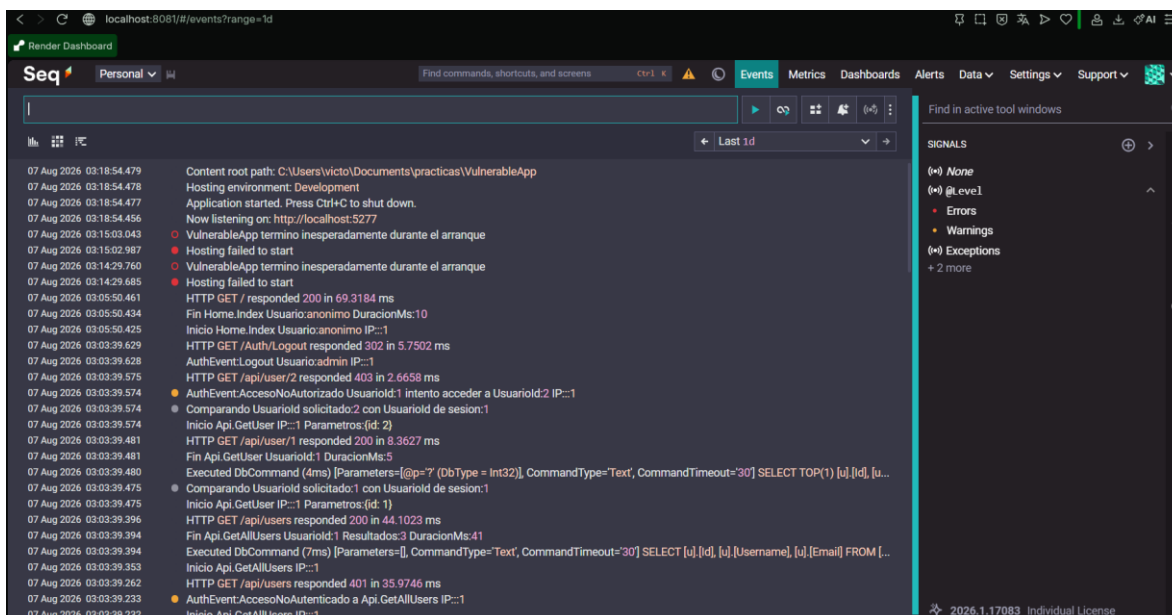
Distribución de eventos por nivel en el archivo correspondiente a la sesión de pruebas:

Nivel	Cantidad
Debug	8
Information	64
Warning	6
Error	5

Los cinco eventos de nivel Error corresponden a la corrida dedicada a provocar una falla real, descrita en la sección 6.6.

6.4 Evidencia del sink de red y de la agregación en Seq

La interfaz web de Seq presenta el flujo de eventos recibidos, con coloreado por nivel y actualización en tiempo real.



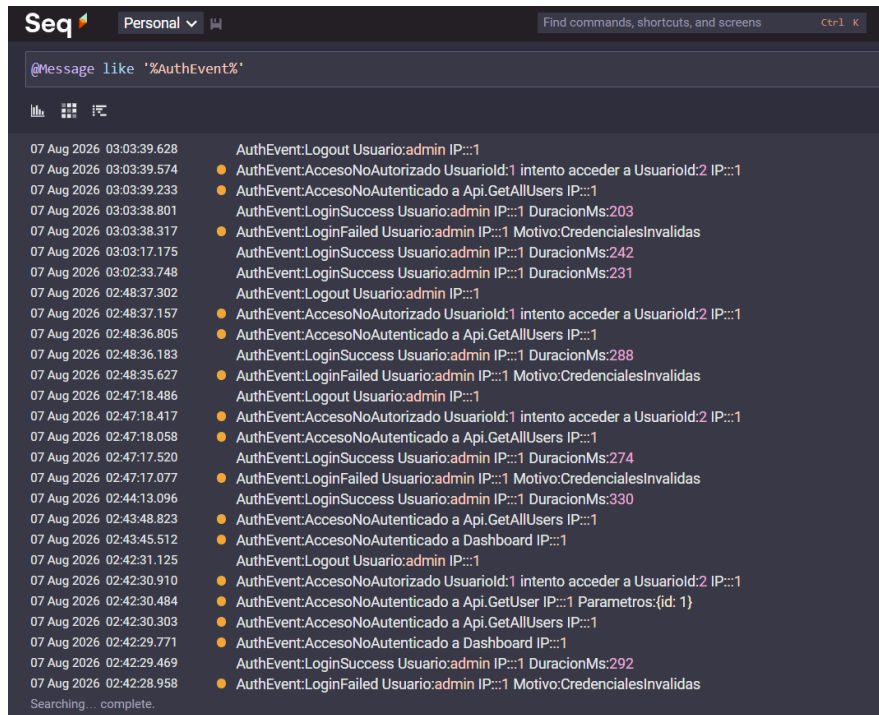
6.5 Consulta filtrada por controlador y por tipo de evento

La capacidad de filtrado constituye la justificación principal de la incorporación de Seq. Las consultas se expresan sobre las propiedades del evento y no sobre el texto renderizado:

@Message like '%Search.Index%'

@Message like '%AuthEvent%'

@Level = 'Warning'



Seq Personal Find commands, shortcuts, and screens Ctrl K

@Message like '%AuthEvent%'

07 Aug 2026 03:03:39.628 AuthEvent:Logout Usuario:admin IP::1

07 Aug 2026 03:03:39.574 ● AuthEvent:AccesoNoAutorizado UsuarioId:1 intento acceder a UsuarioId:2 IP::1

07 Aug 2026 03:03:39.233 ● AuthEvent:AccesoNoAutenticado a Api.GetAllUsers IP::1

07 Aug 2026 03:03:38.801 AuthEvent:LoginSuccess Usuario:admin IP::1 DuracionMs:203

07 Aug 2026 03:03:38.317 ● AuthEvent:LoginFailed Usuario:admin IP::1 Motivo:CredencialesInvalidas

07 Aug 2026 03:03:17.175 AuthEvent:LoginSuccess Usuario:admin IP::1 DuracionMs:242

07 Aug 2026 03:02:33.748 AuthEvent:LoginSuccess Usuario:admin IP::1 DuracionMs:231

07 Aug 2026 02:48:37.302 AuthEvent:Logout Usuario:admin IP::1

07 Aug 2026 02:48:37.157 ● AuthEvent:AccesoNoAutorizado UsuarioId:1 intento acceder a UsuarioId:2 IP::1

07 Aug 2026 02:48:36.805 ● AuthEvent:AccesoNoAutenticado a Api.GetAllUsers IP::1

07 Aug 2026 02:48:36.183 AuthEvent:LoginSuccess Usuario:admin IP::1 DuracionMs:288

07 Aug 2026 02:48:35.627 ● AuthEvent:LoginFailed Usuario:admin IP::1 Motivo:CredencialesInvalidas

07 Aug 2026 02:47:18.486 AuthEvent:Logout Usuario:admin IP::1

07 Aug 2026 02:47:18.417 ● AuthEvent:AccesoNoAutorizado UsuarioId:1 intento acceder a UsuarioId:2 IP::1

07 Aug 2026 02:47:18.058 ● AuthEvent:AccesoNoAutenticado a Api.GetAllUsers IP::1

07 Aug 2026 02:47:17.520 AuthEvent:LoginSuccess Usuario:admin IP::1 DuracionMs:274

07 Aug 2026 02:47:17.077 ● AuthEvent:LoginFailed Usuario:admin IP::1 Motivo:CredencialesInvalidas

07 Aug 2026 02:44:13.096 AuthEvent:LoginSuccess Usuario:admin IP::1 DuracionMs:330

07 Aug 2026 02:43:48.823 ● AuthEvent:AccesoNoAutenticado a Api.GetAllUsers IP::1

07 Aug 2026 02:43:45.512 ● AuthEvent:AccesoNoAutenticado a Dashboard IP::1

07 Aug 2026 02:42:31.125 AuthEvent:Logout Usuario:admin IP::1

07 Aug 2026 02:42:30.910 ● AuthEvent:AccesoNoAutorizado UsuarioId:1 intento acceder a UsuarioId:2 IP::1

07 Aug 2026 02:42:30.484 ● AuthEvent:AccesoNoAutenticado a Api.GetUser IP::1 Parametros:{id: 1}

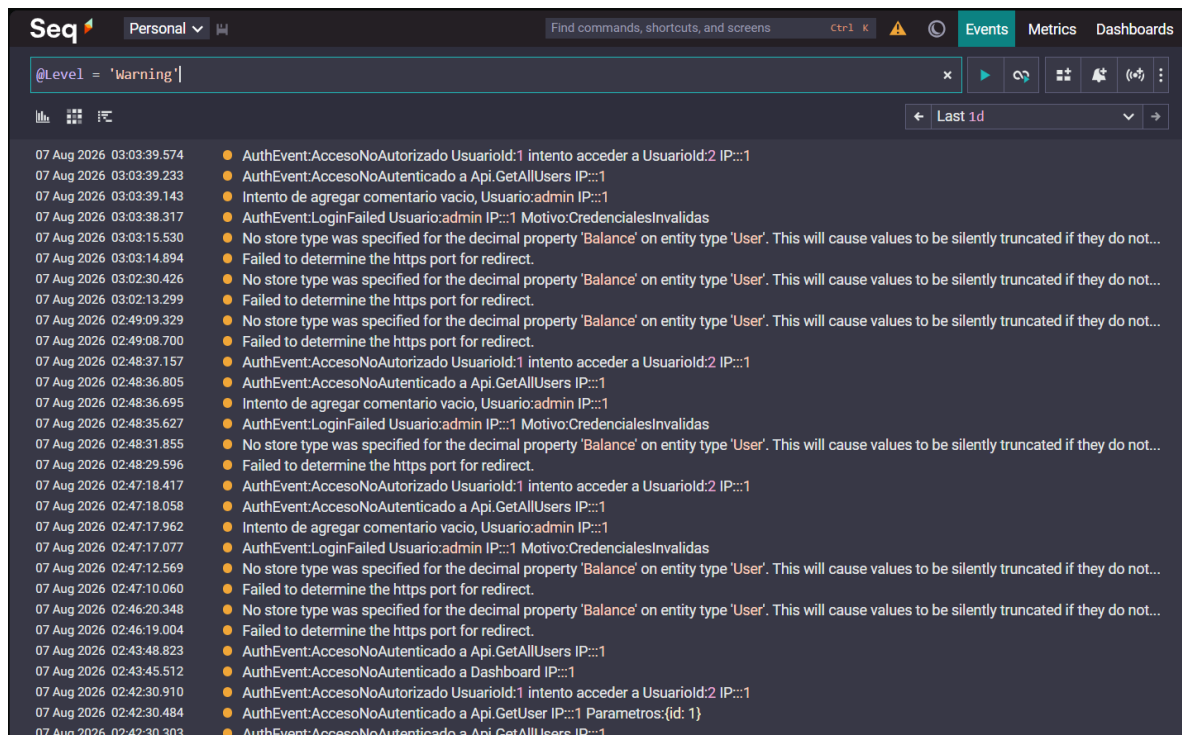
07 Aug 2026 02:42:30.303 ● AuthEvent:AccesoNoAutenticado a Api.GetAllUsers IP::1

07 Aug 2026 02:42:29.771 ● AuthEvent:AccesoNoAutenticado a Dashboard IP::1

07 Aug 2026 02:42:29.469 AuthEvent:LoginSuccess Usuario:admin IP::1 DuracionMs:292

07 Aug 2026 02:42:28.958 ● AuthEvent:LoginFailed Usuario:admin IP::1 Motivo:CredencialesInvalidas

Searching... complete.



Seq Personal Find commands, shortcuts, and screens Ctrl K

@Level = 'Warning'

07 Aug 2026 03:03:39.574 ● AuthEvent:AccesoNoAutorizado UsuarioId:1 intento acceder a UsuarioId:2 IP::1

07 Aug 2026 03:03:39.233 ● AuthEvent:AccesoNoAutenticado a Api.GetAllUsers IP::1

07 Aug 2026 03:03:39.143 ● Intento de agregar comentario vacio, Usuario:admin IP::1

07 Aug 2026 03:03:38.317 ● AuthEvent:LoginFailed Usuario:admin IP::1 Motivo:CredencialesInvalidas

07 Aug 2026 03:03:15.530 ● No store type was specified for the decimal property 'Balance' on entity type 'User'. This will cause values to be silently truncated if they do not...

07 Aug 2026 03:03:14.894 ● Failed to determine the https port for redirect.

07 Aug 2026 03:02:30.426 ● No store type was specified for the decimal property 'Balance' on entity type 'User'. This will cause values to be silently truncated if they do not...

07 Aug 2026 03:02:13.299 ● Failed to determine the https port for redirect.

07 Aug 2026 02:49:09.329 ● No store type was specified for the decimal property 'Balance' on entity type 'User'. This will cause values to be silently truncated if they do not...

07 Aug 2026 02:49:08.700 ● Failed to determine the https port for redirect.

07 Aug 2026 02:48:37.157 ● AuthEvent:AccesoNoAutorizado UsuarioId:1 intento acceder a UsuarioId:2 IP::1

07 Aug 2026 02:48:36.805 ● AuthEvent:AccesoNoAutenticado a Api.GetAllUsers IP::1

07 Aug 2026 02:48:36.695 ● Intento de agregar comentario vacio, Usuario:admin IP::1

07 Aug 2026 02:48:35.627 ● AuthEvent:LoginFailed Usuario:admin IP::1 Motivo:CredencialesInvalidas

07 Aug 2026 02:48:31.855 ● No store type was specified for the decimal property 'Balance' on entity type 'User'. This will cause values to be silently truncated if they do not...

07 Aug 2026 02:48:29.596 ● Failed to determine the https port for redirect.

07 Aug 2026 02:47:18.417 ● AuthEvent:AccesoNoAutorizado UsuarioId:1 intento acceder a UsuarioId:2 IP::1

07 Aug 2026 02:47:18.058 ● AuthEvent:AccesoNoAutenticado a Api.GetAllUsers IP::1

07 Aug 2026 02:47:17.962 ● Intento de agregar comentario vacio, Usuario:admin IP::1

07 Aug 2026 02:47:17.077 ● AuthEvent:LoginFailed Usuario:admin IP::1 Motivo:CredencialesInvalidas

07 Aug 2026 02:47:12.569 ● No store type was specified for the decimal property 'Balance' on entity type 'User'. This will cause values to be silently truncated if they do not...

07 Aug 2026 02:47:10.060 ● Failed to determine the https port for redirect.

07 Aug 2026 02:46:20.348 ● No store type was specified for the decimal property 'Balance' on entity type 'User'. This will cause values to be silently truncated if they do not...

07 Aug 2026 02:46:19.004 ● Failed to determine the https port for redirect.

07 Aug 2026 02:43:48.823 ● AuthEvent:AccesoNoAutenticado a Api.GetAllUsers IP::1

07 Aug 2026 02:43:45.512 ● AuthEvent:AccesoNoAutenticado a Dashboard IP::1

07 Aug 2026 02:42:30.910 ● AuthEvent:AccesoNoAutorizado UsuarioId:1 intento acceder a UsuarioId:2 IP::1

07 Aug 2026 02:42:30.484 ● AuthEvent:AccesoNoAutenticado a Api.GetUser IP::1 Parametros:{id: 1}

07 Aug 2026 02:42:30.303 ● AuthEvent:AccesoNoAutenticado a Api.GetAllUsers IP::1

Los eventos recuperados en esta última consulta poseen valor de seguridad directo: constituyen exactamente el conjunto que un monitoreo operativo vigilaría para detectar intentos de acceso no autorizado, y su concentración bajo un prefijo común permite construir sobre ellos una alerta automática.

6.6 Prueba de generación de error real

Se provocó una falla de conectividad con el motor de base de datos, redirigiendo la cadena de conexión hacia una instancia inexistente, y se emitió una petición al controlador de búsqueda. La falla quedó capturada de forma íntegra en los tres sinks:

[ERR] An error occurred using the connection to database 'VulnerableDb' on server '(localdb)\instancia_inexistente_test'.

[ERR] An exception occurred while iterating over the results of a query for context type

'VulnerableApp.Data.AppDbContext'.

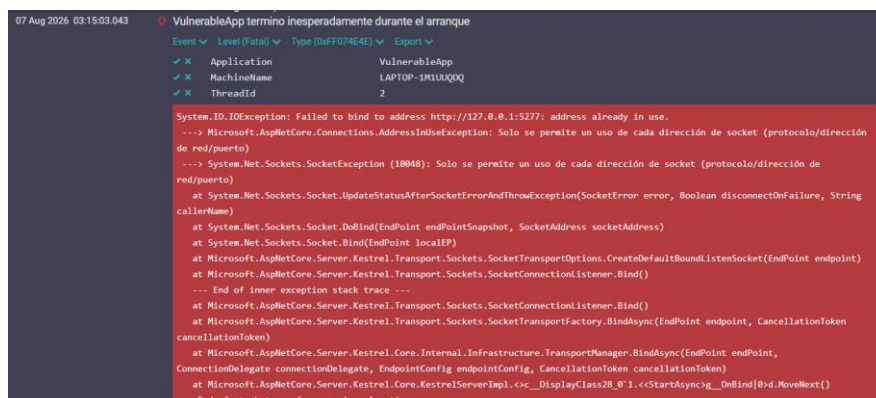
[ERR] Error en Search.Index Usuario:anonimo IP:::1 Parametros:{"search":"admin"}

[ERR] HTTP GET /Search/Index responded 500 in 3700.1909 ms

[ERR] An unhandled exception has occurred while executing the request.

La secuencia resulta ilustrativa de cómo se estratifica el registro en una arquitectura por capas. La tercera línea corresponde al LogError explícito incorporado en el controlador, único evento que aporta contexto de negocio —qué usuario, desde qué dirección y con qué parámetros—; las restantes son generadas automáticamente por Entity Framework Core, por el middleware de registro de peticiones y por el manejador global de excepciones. La lectura conjunta permite reconstruir la falla desde el síntoma observable por el cliente hasta su causa técnica en la capa de datos.

Merece atención la duración registrada, cercana a los 3,7 segundos: no corresponde al tiempo de procesamiento sino al agotamiento del temporizador de conexión del proveedor de datos. Este dato, que ningún registro no instrumentado habría capturado, permite distinguir una falla por indisponibilidad del servidor de una falla por credenciales incorrectas o por consulta malformada, que se manifestarían de inmediato.



```
07 Aug 2026 03:15:03.043 VulnerableApp termino inesperadamente durante el arranque
Event Level(Fatal) Type (0x00000000) Export
Application VulnerableApp
MachineName LAPTOP-3M1UJQDQ
ThreadId 2

System.IO.IOException: Failed to bind to address http://127.0.0.1:5277: address already in use.
----> Microsoft.AspNetCore.Connections.AddressInUseException: Solo se permite un uso de cada dirección de socket (protocolo/dirección de red/puerto)
----> System.Net.Sockets.SocketException (10048): Solo se permite un uso de cada dirección de socket (protocolo/dirección de red/puerto)
at System.Net.Sockets.Socket.UpdateStatusAfterSocketErrorAndThrowException(SocketError error, Boolean disconnectOnFailure, String callerName)
at System.Net.Sockets.Socket.DoBind(EndPoint endPointSnapshot, SocketAddress socketAddress)
at System.Net.Sockets.Socket.Bind(EndPoint localEP)
at Microsoft.AspNetCore.Server.Kestrel.Transport.Sockets.SocketTransportOptions.CreateDefaultBoundListenSocket(EndPoint endPoint)
at Microsoft.AspNetCore.Server.Kestrel.Transport.Sockets.SocketConnectionListener.Bind()
--- End of inner exception stack trace ---
at Microsoft.AspNetCore.Server.Kestrel.Transport.Sockets.SocketConnectionListener.Bind()
at Microsoft.AspNetCore.Server.Kestrel.Transport.Sockets.SocketTransportFactory.BindAsync(EndPoint endPoint, CancellationToken cancellationToken)
at Microsoft.AspNetCore.Server.Kestrel.Core.Internal.Infrastructure.TransportManager.BindAsync(EndPoint endPoint, ConnectionDelegate connectionDelegate, EndpointConfig endpointConfig, CancellationToken cancellationToken)
at Microsoft.AspNetCore.Server.Kestrel.Core.KestrelServerImpl.<_DisplayClass28_0`1.><StartAsync>g__OnBind|0.d.MoveNext()
--- End of stack trace from previous location ---
```

6.7 Verificación empírica del control de nivel desde configuración

Para comprobar de forma reproducible que el nivel mínimo declarado en configuración gobierna efectivamente el registro, se ejecutó **la misma secuencia de tres peticiones** —búsqueda, autenticación y consulta a la API— en dos ocasiones, modificando exclusivamente la variable ASPNETCORE_ENVIRONMENT y sin alterar una sola línea de código.

Entorno	Nivel efectivo	mínimo Eventos Debug	Eventos Information	Eventos Warning
Production	Information	0	16	2
Development	Debug	3	16	2

Los tres eventos que aparecen únicamente bajo el entorno de desarrollo son:

[DBG] Ejecutando consulta parametrizada Users.Where(Username contains admin)

[DBG] Verificando hash BCrypt para Usuario:admin (password nunca se registra en logs)

[DBG] Comparando Usuariold solicitado:1 con Usuariold de sesion:1

El resultado demuestra que el filtrado opera conforme a lo previsto, y su diseño experimental permite atribuir la diferencia exclusivamente a la variable modificada: la invariancia de los conteos de Information y Warning entre ambas corridas confirma que la secuencia de peticiones fue idéntica y que el único factor alterado fue el umbral de registro.

El contenido de estos tres eventos resulta además significativo desde la perspectiva de la materia. Los tres documentan la ejecución de un control de seguridad —parametrización de la consulta, verificación del hash de la credencial y comprobación de pertenencia del recurso—, es decir, exactamente las mitigaciones implementadas en la rama secure del proyecto. Su registro en nivel Debug constituye la elección adecuada: aportan valor durante la depuración y la auditoría de la implementación, pero su volumen y su detalle los tornan inapropiados para operación continua.

6.8 Verificación de ausencia de credenciales en los registros

Se ejecutó una búsqueda insensible a mayúsculas sobre el archivo de registro, empleando tanto la contraseña incorrecta utilizada deliberadamente en la prueba de autenticación fallida como la contraseña correcta empleada en la prueba de autenticación exitosa:

```
bash
```

```
grep -icE "incorrect|password|admin" Logs/log-20260807.txt
```

```
# resultado: 0
```

El resultado nulo confirma que **ninguna de las dos credenciales aparece en el registro**, y que únicamente se persiste el nombre de usuario. La prueba verifica que el código del controlador de autenticación no interpola en ningún punto la variable de contraseña dentro de una plantilla de mensaje.

Conviene subrayar el valor metodológico de esta comprobación. La ausencia de credenciales en los registros constituye una propiedad que la revisión visual del código puede fácilmente pasar por alto, dado que basta una única instrucción de registro descuidada, situada en una ruta de error poco frecuentada, para invalidarla. La verificación mediante búsqueda sobre el registro efectivamente producido examina el resultado observable en lugar de la intención del código, y por ello constituye evidencia de naturaleza más fuerte.

6.9 Verificación mediante la API de Seq

Como evidencia complementaria a las capturas de la interfaz, se consultó directamente la API del servidor:

```
GET http://localhost:5341/api/events?count=2000
```

Resultados agregados:

Métrica	Valor
Eventos totales recibidos	349
Nivel Information	295
Nivel Warning	34
Nivel Error	10
Nivel Debug	8
Nivel Fatal	2

Eventos con propiedad Application = VulnerableApp 349 (100 %)

La cobertura del 100 % en la propiedad Application confirma que la configuración de propiedades globales se aplica de manera efectiva a la totalidad del flujo, sin excepciones ni eventos emitidos por fuera del pipeline configurado.

Precisión sobre la diferencia entre los conteos. Los totales obtenidos de Seq superan a los del archivo consignados en la sección 6.3, y la discrepancia es esperada y explicable. El archivo rota a diario y refleja la sesión de pruebas concreta, mientras que Seq acumula de forma persistente los eventos de todas las corridas realizadas durante la práctica, incluidas las de desarrollo y depuración previas a las pruebas finales. Ambos valores son correctos dentro de su respectivo alcance temporal, y su comparación ilustra la diferencia funcional entre un registro rotativo local y un repositorio centralizado de historial.

7. Hallazgos de troubleshooting

Durante el desarrollo de la práctica se identificaron y resolvieron dos condiciones no previstas en la guía, que se documentan por su valor formativo.

7.1 Imposibilidad de arranque de Seq con el comando de la guía

Conforme se detalla en la sección 4.2, la imagen actual de Seq rechaza el arranque cuando no se define contraseña de administrador ni se desactiva explícitamente la autenticación. La combinación de este comportamiento con la directiva `--restart unless-stopped` genera un síntoma potencialmente confuso: el contenedor aparece de forma reiterada en la lista de contenedores activos, en estado `Restarting`, lo que puede interpretarse erróneamente como un proceso en curso de inicialización.

El diagnóstico requirió inspeccionar los registros del contenedor con `docker logs seq`, donde el error aparece de forma explícita. La lección procedimental es directa: ante un contenedor que no responde, la primera acción diagnóstica corresponde a la revisión de sus registros y no a la repetición del comando de arranque.

7.2 Eventos de nivel Fatal falsos generados por las herramientas de Entity Framework Core

Los dos eventos de nivel Fatal registrados en Seq corresponden a ejecuciones de `dotnet ef database update` realizadas **antes** de incorporar la cláusula `when (ex is not HostAbortedException)` descrita en la sección 4.3.

El mecanismo subyacente es el siguiente: las herramientas de diseño de Entity Framework Core construyen el host de la aplicación para inspeccionar los servicios registrados y, una vez obtenida esa información, lo abortan de forma deliberada lanzando `HostAbortedException`. Se trata de un mecanismo de control de flujo previsto por el framework, no de una falla. Sin embargo, el bloque `catch` genérico del patrón de bootstrap logger la interceptaba y la registraba con la máxima severidad.

El impacto de este falso positivo no es meramente cosmético. Un registro que contiene eventos de severidad Fatal originados en operaciones rutinarias erosiona

la confianza en la clasificación por niveles y, en un entorno con alertas automatizadas, generaría notificaciones espurias que conducen al fenómeno de fatiga de alertas, en el cual el operador termina por ignorar precisamente las notificaciones que debería atender.

Tras la incorporación del filtro, se verificó que las ejecuciones subsecuentes de comandos de Entity Framework Core no vuelven a producir eventos de este tipo. Ambos hallazgos quedan por tanto documentados como incidencias resueltas y no como pendientes.

8. Tabla resumen de cumplimiento

Métrica	Valor
Controladores instrumentados	5 de 5 (Home, Search, Auth, Comment, Api)
Sinks configurados	3 de 3 (Console, File, Seq)
Enrichers configurados	3 (FromLogContext requerido, más WithMachineName y WithThreadId justificados)
Eventos totales recibidos en Seq	349
Eventos de nivel Information	295
Eventos de nivel Warning	34
Eventos de nivel Error	10
Eventos de nivel Debug	8
Cobertura de la propiedad Application	100 %
Credenciales detectadas en los registros	0
Diferencial de eventos entre entornos	Debug 0 en Production frente a 3 en Development, sobre secuencia idéntica
Eventos de autenticación implementados	LoginSuccess, LoginFailed, Logout, AccesoNoAutenticado, AccesoNoAutorizado

Métrica	Valor
Política de rotación de archivo	Diaria, verificada
Hallazgos de troubleshooting documentados y resueltos	2

9. Conclusiones

Se configuró Serilog con tres sinks y tres enrichers, con el nivel mínimo gobernado íntegramente desde los archivos de configuración y sin dependencia de constantes en el código. Los cinco controladores requeridos quedaron instrumentados con `ILogger<T>`, registrando entrada, salida, duración, identidad, dirección de origen, parámetros, advertencias, errores, eventos de autenticación y excepciones, y la implementación se validó mediante tráfico real y no únicamente por revisión de código.

El aporte central de la práctica reside en la distinción entre registro textual y registro estructurado. La conservación de la plantilla del mensaje y de las propiedades como entidades independientes convierte al registro de un artefacto destinado a la lectura humana en un conjunto de datos consultable, sobre el cual pueden construirse filtros, agregaciones y alertas. La consulta por nivel de advertencia, que recupera de forma inmediata la totalidad de los intentos de acceso rechazados con independencia del controlador que los originó, ilustra con precisión esa diferencia.

Desde la perspectiva de la seguridad, el registro constituye simultáneamente un control y un riesgo. Es un control porque hace observable la actividad del sistema y habilita la detección de patrones de ataque que ninguna revisión de código podría anticipar; es un riesgo porque concentra en un artefacto de menor protección información que la aplicación custodia con cuidado. La verificación explícita de que ninguna credencial alcanza los registros atiende precisamente esa segunda dimensión, y su comprobación empírica sobre el registro efectivamente producido —en lugar de una revisión visual del código— constituye evidencia de naturaleza más robusta.

Los dos hallazgos de troubleshooting aportan una lección adicional. Tanto la exigencia de contraseña de administrador en Seq como el falso positivo de severidad Fatal derivado de las herramientas de Entity Framework Core corresponden a comportamientos no documentados en la guía, resueltos mediante lectura de registros y comprensión del mecanismo subyacente en lugar de ensayo y error. El segundo caso ilustra además una propiedad importante del propio sistema de registro: un evento clasificado con severidad incorrecta degrada el valor

de todo el registro, porque la utilidad de la clasificación por niveles depende íntegramente de su fiabilidad.